Human Verification: `Containers.ProofSearch`

Binary Proof Search Trees as a Directed Container

Machine-generated from `lean/ProofSearch.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.ProofSearch` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 Shapes: binary search trees **2**
2.1 Inductive type: `STree` 2

3 Positions: paths in a tree **2**
3.1 Inductive type: `SPos` 2

4 The directed-container operations **3**
4.1 Definition: `STree.sub` 3
4.2 Definition: `STree.shift` 3

5 Directed-container laws **4**
5.1 Theorem: `STree.shift_root` (D3) 4
5.2 Theorem: `STree.sub_shift` (D4) 4
5.3 Theorem: `STree.shift_assoc` (D5) 4

6 The directed container **5**
6.1 Definition: `proofSearch` 5

7 Translation conventions **6**

1 Overview

This file realises binary proof-search trees as a directed container. The shapes are binary tree structures; positions are paths from the root to any node. The directed-container comonad structure comes from *path concatenation*:

- $\mathrm{root}(t) = \varepsilon$ (the empty path `here`);
- $t \downarrow p =$ the sub-tree rooted at position p;

- $p \oplus q$ = the path obtained by following p then q.

Laws D1 and D2 hold by rfl (definitionally from the first cases of the recursive definitions). D3–D5 are verified by structural recursion on paths.

The induced comonad on Ext(proofSearch) has:

- extract$(t, v) = v(\varepsilon)$: read the label at the root;
- duplicate$(t, v) = (t,\ p \mapsto (t \downarrow p,\ q \mapsto v(p \oplus q)))$: at every node, replace the label with the full sub-search from that point.

A coKleisli morphism $W(X) \to Y$ is a **proof strategy**: a function that inspects the entire search tree to decide what to produce.

2 Shapes: binary search trees

2.1 Inductive type: `STree`

Definition 2.1 (Binary search tree)**.** A **binary search tree shape** is either a `leaf` (a terminal node) or a `node` with two sub-trees:

$$\text{STree} ::= \texttt{leaf} \mid \texttt{node}(\ell, r).$$

A leaf represents either a dead end or a success state; the distinction lives in the labelling, not the shape.

Lean.
```
inductive STree where
  | leaf : STree
  | node : STree → STree → STree
```

Haskell.
```
data STree = Leaf | Node STree STree
  deriving (Show, Eq)
```

3 Positions: paths in a tree

3.1 Inductive type: `SPos`

Definition 3.1 (Position in a binary search tree)**.** A **position** in an `STree` is a path from the root to a node:

- `here`: the root (valid for any tree);
- `left` p: descend into the left sub-tree and continue along p (only valid for a `node`);
- `right` p: similarly for the right sub-tree.

For a `leaf`, the only position is `here` — this is what makes a leaf a dead end.

Lean.
```
inductive SPos : STree → Type where
  | here  : SPos t
  | left  : SPos l → SPos (.node l r)
  | right : SPos r → SPos (.node l r)
```

Haskell.
```haskell
data SPos t where
  Here  :: SPos t
  Left  :: SPos l → SPos (Node l r)
  Right :: SPos r → SPos (Node l r)
-- Requires GADTs or can be approximated as:
data SPos = Here | GoLeft SPos | GoRight SPos
```

Fidelity note. In Lean, `SPos` is indexed by the tree: `SPos t` is the type of positions *in* t. The `left` constructor is only well-typed when the tree is a `node`. In plain Haskell, the index is not enforced; `GoLeft Here` on a leaf would type-check but be semantically invalid. The GADT version recovers the index.

4 The directed-container operations

4.1 Definition: `STree.sub`

Definition 4.1 (Sub-tree at a position)**.** The sub-tree at position p in tree t:

$$\begin{aligned} t \downarrow \texttt{here} &= t, \\ (\texttt{node}\, \ell\, r) \downarrow \texttt{left}(p) &= \ell \downarrow p, \\ (\texttt{node}\, \ell\, r) \downarrow \texttt{right}(p) &= r \downarrow p. \end{aligned}$$

Lean.
```
def STree.sub : (t : STree) → SPos t → STree
  | t, .here ⇒ t
  | .node l _, .left p ⇒ l.sub p
  | .node _ r, .right p ⇒ r.sub p
```

Haskell.
```haskell
sub :: STree → SPos → STree
sub t    Here        = t
sub (Node l _) (GoLeft  p) = sub l p
sub (Node _ r) (GoRight p) = sub r p
sub Leaf   _         = error "invalid position"
```

4.2 Definition: `STree.shift`

Definition 4.2 (Shift (path concatenation))**.** The shift $p \oplus q$ embeds position q in the sub-tree at p back into the whole tree — this is path concatenation:

$$\begin{aligned} t.\texttt{shift}(\texttt{here}, q) &= q, \\ (\texttt{node}\, \ell\, r).\texttt{shift}(\texttt{left}(p), q) &= \texttt{left}(\ell.\texttt{shift}(p, q)), \\ (\texttt{node}\, \ell\, r).\texttt{shift}(\texttt{right}(p), q) &= \texttt{right}(r.\texttt{shift}(p, q)). \end{aligned}$$

Lean.
```
def STree.shift : (t : STree) → (p : SPos t) → SPos (t.sub p) →
    SPos t
  | _, .here, q ⇒ q
  | .node l _, .left p, q ⇒ .left (l.shift p q)
  | .node _ r, .right p, q ⇒ .right (r.shift p q)
```

Haskell.
```haskell
shift :: STree → SPos → SPos → SPos
shift _           Here        q = q
shift (Node l _) (GoLeft  p) q = GoLeft  (shift l p q)
```

```
shift (Node _ r) (GoRight p) q = GoRight (shift r p q)
shift Leaf       _           _ = error "invalid␣position"
```

5 Directed-container laws

5.1 Theorem: `STree.shift_root` (D3)

Theorem 5.1 (D3: right identity)**.** Shifting by the root of the sub-tree recovers the original position (right unit law: $p \oplus \varepsilon = p$):

$$t.\texttt{shift}(p, \texttt{here}) \;=\; p \quad \text{for all } p : \mathrm{SPos}(t).$$

Lean.
```
def STree.shift_root : (t : STree) → (p : SPos t) →
    t.shift p .here = p
```

Haskell.
```
-- D3 (right identity):
-- prop_shift_root t p = shift t p Here == p
```

5.2 Theorem: `STree.sub_shift` (D4)

Theorem 5.2 (D4: sub-trees nest along the shift)**.** The sub-tree at a concatenated path equals descending twice:

$$t \downarrow (p \oplus q) \;=\; (t \downarrow p) \downarrow q \quad \text{for all } p : \mathrm{SPos}(t),\; q : \mathrm{SPos}(t \downarrow p).$$

Lean.
```
def STree.sub_shift : (t : STree) → (p : SPos t) → (q : SPos (t.sub
    p)) →
     t.sub (t.shift p q) = (t.sub p).sub q
```

Haskell.
```
-- D4:
-- prop_sub_shift t p q = sub t (shift t p q) == sub (sub t p) q
```

5.3 Theorem: `STree.shift_assoc` (D5)

Theorem 5.3 (D5: shift is associative)**.** Path concatenation is associative (with transport along D4):

$$t.\texttt{shift}(p \oplus q,\; D4^{-1} \triangleright r) \;=\; t.\texttt{shift}(p,\; (t \downarrow p).\texttt{shift}(q, r))$$

for all $p : \mathrm{SPos}(t)$, $q : \mathrm{SPos}(t \downarrow p)$, $r : \mathrm{SPos}((t \downarrow p) \downarrow q)$.

The transport $D4^{-1} \triangleright r$ is needed because r lives over $(t \downarrow p) \downarrow q$, while the left-hand side needs it over $t \downarrow (p \oplus q)$; D4 witnesses that these two types are equal.

Lean.
```
def STree.shift_assoc : (t : STree) → (p : SPos t) → (q : SPos (t.
    sub p)) →
     (r : SPos ((t.sub p).sub q)) →
     t.shift (t.shift p q) ((t.sub_shift p q).symm [> r) =
       t.shift p ((t.sub p).shift q r)
```

Haskell.

```haskell
-- D5 (associativity, transport suppressed):
-- prop_shift_assoc t p q r =
--   shift t (shift t p q) r == shift t p (shift (sub t p) q r)
```

Fidelity note. The Lean statement requires r to be transported along sub_shift (D4) from the fibre over $(t \downarrow p) \downarrow q$ to the fibre over $t \downarrow (p \oplus q)$. In Haskell, positions are unindexed and the transport is invisible.

6 The directed container

6.1 Definition: `proofSearch`

Definition 6.1 (The `proofSearch` directed container)**.** Define the directed container of binary proof-search trees as:

$$\text{proofSearch} \;=\; (\text{STree},\ \text{SPos},\ \texttt{here},\ \texttt{STree.sub},\ \texttt{STree.shift})$$

with:

- D1: $t \downarrow \texttt{here} = t$ (by rfl);
- D2: $t.\texttt{shift}(\texttt{here}, q) = q$ (by rfl);
- D3: `STree.shift_root`;
- D4: `STree.sub_shift`;
- D5: `STree.shift_assoc`.

Lean.

```lean
def proofSearch : DirectedContainer where
  Shape := STree
  Pos := SPos
  root := fun _ ⇒ .here
  sub := STree.sub
  shift := STree.shift
  d1 := fun _ ⇒ rfl
  d4 := STree.sub_shift
  d2 := fun _ _ ⇒ rfl
  d3 := STree.shift_root
  d5 := STree.shift_assoc
```

Haskell.

```haskell
proofSearch :: DirectedContainer STree SPos
proofSearch = DirectedContainer
  { shape = STree
  , pos   = SPos
  , root  = const Here
  , sub   = sub
  , shift = shift
  -- D1: sub t Here = t                (by definition)
  -- D2: shift t Here q = q            (by definition)
  -- D3: shift t p Here = p            (STree.shift_root)
  -- D4: sub t (shift t p q) = sub (sub t p) q  (STree.sub_shift)
  -- D5: shift t (shift t p q) r = shift t p (shift (sub t p) q r)  (
     D5)
  }
```

7 Translation conventions

1. **D1 and D2 are definitional.** The first case in both `STree.sub` and `STree.shift` handles `here`, giving $t \downarrow \texttt{here} = t$ and $t.\texttt{shift}(\texttt{here}, q) = q$ by rfl. D3–D5 require structural recursion.
2. **Positions are indexed.** In Lean, `SPos t` is the type of positions in t. The tree structure constrains which constructors are valid. Haskell approximates this with an unindexed `SPos` type.
3. **The D5 transport.** D5 requires a position to be transported along D4 before shift can consume it. This transport is rfl in both the `here` and `left`/`right` cases (the recursion is definitional), but it must be stated in the type of D5 to make the law well-typed.
4. **Comonad reading.** The induced comonad on Ext(proofSearch) is defined by the general machinery of directed containers; its laws follow from D1–D5. This document only formalises the directed-container structure itself.
5. **Proof bodies omitted.** Only statements are translated.